

Week 1 - Day 2b: Lists, Strings, Dictionaries, Loops

You will be working on a series of exercises for the material covered in lecture 2b, including:

- **Lists** (Assignment, Adding, Multiplying, Indexing, Slicing)
- **Strings**
- **Dictionaries**
- **Loops**

We will go through a series of exercises to reinforce the concepts covered today.

Exercise 1: Lists []

1.1 (Creating Lists) Create a list containing the number 5731, the string "bye", and the boolean True.

```
# TODO: Your code here
```

1.2 (List Addition) Define two lists, one containing the numbers 87, 109, 3 and the other containing the strings ["string1", "string2", "string3"]. Call these list1 and list2 respectively. Then, add the two lists together.

```
# TODO: Your code here
```

1.3 (List Addition) What happens when you switch the order of the addition? Write your answer before you try it out in code.

```
# TODO: Your code here
```

1.4 (List Multiplication) Suppose list1 is [1, 2, 3]. Use list multiplication to create a new list list2, which is [1, 2, 3, 1, 2, 3].

```
# TODO: Your code here
```

1.5 (List Indexing) Suppose that lst = [2, 5, 2, 5, 9, 7], what is lst[2]? Write your answer below before testing in code.

```
lst = [2, 5, 2, 5, 9, 7]
```

```
# TODO: Your code here
```

1.6 (Length of Lists) Use the len function to find the length of a string s = "hello" and a list lst = [2, 3, 45, "t5", "ew", 4]

```
s = "hello"
lst = [2, 3, 45, "t5", "ew", 4]

# TODO: Your code here
```

1.7 (List Indexing) Suppose that you are given a sorted list `lst = [0, 2, 3, 5, 7, 21, 837, 3284, 23842934]`, what is the median of this list? Use `len(lst)` to find the length of this list.

```
lst = [0, 2, 3, 5, 7, 21, 837, 3284, 23842934]

# TODO: Your code here
```

1.8 (List Slicing) Given the list `lst = [2, 3, 4, 5, 6]` of odd length, print out the middle three numbers in a new list.

```
lst = [2, 3, 4, 5, 6]

# TODO: Your code here
```

1.9 (Nested Lists) Suppose you have a nested list `lst = [["addis", 1, 3, ["coder"]], [237894, "ajhfoiah"]]`. What's the type of `lst[0]`?

```
lst = [["addis", 1, 3, ["coder"]], [237894, "ajhfoiah"]]
```

1.10 (Nested List) Given the same nested list, `lst`, what is the value of `lst[0][0]`?

1.11 (Nested List) Given the same nested list, `lst`, print out the element `"coder"` from this list.

```
# TODO: Your code here
```

You can put lists inside of lists to create a matrix, like:

```
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

to represent

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

```
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

1.12 (Matrices) How would you access the entry in the first row, first column (whose value is 1)?

```
# TODO: Your code here
```

1.13 (Matrices) How would you access the entry whose value is 8?

```
# TODO: Your code here
```

1.14 You are given two **lists of strings** called `names` and `cities`. The **first person** in `names` lives in the **last city** in `cities`, the second person lives in the second to last city, the third lives in the third to last, and so on. Write code to create a third list called `inOrder` such that the even indexes (0, 2, 4, etc.) are the names of the people (in order) and that the odd indexes (1, 3, 5, etc.) are the cities. For a person in index i , their corresponding city should be in index $i+1$.

For example:

```
names = ["Prof. Daniel", "Prof. Huy", "Prof. Jelani", "Prof. Huacheng"]
cities = ["Princeton", "Cambridge", "Boston", "San Francisco"]
```

In this case,

- Prof. Daniel lives in San Francisco
- Prof. Huy lives in Boston
- Prof. Jelani lives Cambridge
- Prof. Haucheng lives in Princeton

You should create the final list:

```
inOrder = ["Prof. Daniel", "San Francisco", "Prof. Huy", "Boston",
"Prof. Jelani", "Cambridge", "Prof. Huacheng", "Princeton"]

# TODO: Your code here
names = ["Timnit", "Binam", "Basi"]
cities = ["Harar", "Addis Ababa", "Gondar"]
```

Exercise 2: Strings []

2.1 Given a string `st = "addis coder"`, find the second to the last character.

```
st = "addis coder"

# TODO: Your code here
```

2.2 Change the second to the last char of the above `st` to `'i'`, and see if it is possible.

```
# TODO: Your code here
```

2.3 Given a string `st = "ddkang"`, remove the first two letters of `st` to create a new string `str1`. Create a second string `str2` from `str1` so that `str2` is `"daniel kang"`

```
st = "ddkang"

# TODO: Your code here
```

Exercise 3: Dictionaries [] []

3.1 Create a dictionary `dic` with the following key-value pairs.

Key	Value
Daniel	San Francisco
Huy	Boston
Jelani	Cambridge
Huacheng	Princeton

```
# TODO: Your code here
```

3.2 Given the dictionary `dic` from above, find "Jelani"'s city.

```
# TODO: Your code here
```

3.3 Given the dictionary `dic`, change "Jelani"'s city to "Berkeley".

```
# TODO: Your code here
```

3.4 Find the length of `dic`.

```
# TODO: Your code here
```

3.5 Print the keys of `dic` as a list

```
# TODO: Your code here
```

Exercise 4: Loops [] [] []

! If your code does not stop running and has an `[*]` on the left, it is in an infinite loop. To get out of this loop, in the top menu, select Kernel -> Interrupt, or press the black square button in the top menu bar.

4.1 What happens with the code inside the loop if you write a `while True:` loop? Explain below!

Try running the code below and see what happens

Hint: You might need to use the stop button at the top of the page

```
i = 0
while True:
    i += 1
    print(i)
```

4.2 What happens with the code inside the loop if you write a `while False:` loop?

4.3 Use a while loop to print the numbers between 0 to 10 (inclusive on both ends).

```
# TODO: Your code here
```

4.4 Use a while loop to print the even numbers between 0 to 10 (inclusive on both ends).

```
# TODO: Your code here
```

4.5 When computing $1+2+3+4+\dots$, how many numbers do you need to sum up so that the sum exceeds 1000?

Hint: Use a while loop

```
# TODO: Your code here
```

4.6 Let $n = 345323$. Check if n is prime. Print "n is prime" if it is, or "n is not prime" otherwise.

Hint: Use a while loop

```
# TODO: Your code here
```

4.7 Suppose that you have `lst = [1, 7, 4, 5, 10, 2, 3, 6, 9, 12]`. Use a while loop to find the first number that is at least 10.

```
# TODO: Your code here
```

4.8 Suppose that you have `lst = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]`, reverse the list using a while loop.

```
# TODO: Your code here
```

4.9 The **Collatz sequence** of a number n is defined as follows:

- It starts with the number $a_0 = n$ itself
- If the i -th number a_i is even, then the next number a_{i+1} is $\frac{a_i}{2}$
- If the i -th number a_i is odd, then the next number a_{i+1} is $3 \cdot a_i + 1$
- The sequence ends once you reach the number 1.

For example, for $n=12$, the sequence is: 12, 6, 3, 10, 5, 16, 8, 4, 2, 1.

The Collatz conjecture says that whichever number you start with, you'll always reach 1 at some point.

Define an integer variable n and then use a while loop to print out its Collatz sequence!

```
# TODO: Your code here
```

4.11 The Fibonacci sequence is defined as follows: The first two elements in the sequence $f(0)$ and $f(1)$ are equal to 1. For every $n > 1$, the n -th element in the sequence is

$$f(n) = f(n-1) + f(n-2).$$

Use a while loop to write a code that prints out all the Fibonacci numbers less than 10000.

```
# TODO: Your code here
```

Exercise 5: Find the Mistake []]

In each of the following code snippets, there is a mistake. Find it and fix it!

Suggestion: First run the code and look at the error message. Fix it and run it again.

5.1 The code below should sum the first 10 square numbers, but the loop just runs infinitely. Find the mistake and fix it.

```
# TODO: Find the mistake and fix it
i = 1
sum = 0
while i <= 10:
    sum += i**2
print(sum)
```

5.2 The code below should sum the first few numbers until their sum exceeds 100, but nothing is printed out. Find the mistake and fix it.

```
# TODO: Find the mistake and fix it
total = 0
i = 1
while total >= 100:
    total += i
    i += 1
print(i)
```

5.3 The code below should print the first number in `lst` that is bigger than 10. However, the output of the code is 5 which is certainly not bigger than 10. Find the mistake and fix it.

```
# TODO: Find the mistake and fix it
lst = [1,3,9,5,-1,13,5,11,2]
number = 0
while lst[number] <= 10:
    number += 1
print(number)
```

Exercise 6: Translate the sentence []

Litus often writes messages on his phone. But sometimes the words he writes get translated into a strange language. You are given a list of words that Litus wrote, and a dictionary from his language to English. Your task is to find out what Litus is trying to say, by replacing each word that is present as a key in the dictionary.

Example: if `words` is `["are", "nosotros", "going", "a", "comer", "lunch"]`, then the answer is `["are", "we", "going", "to", "eat", "lunch"]`. Note that `lunch` is not present in the dictionary `esp_to_eng`, so it does not get replaced.

```
esp_to_eng = {"a": "to", "nosotros": "we", "piña": "pineapple",  
"comer": "eat", "patata": "potato", "felicidades": "congratulations",  
"el": "the", "y": "and", "suerte": "luck", "ejercicios": "exercises"}  
  
words = ["felicidades", "y", "good", "suerte", "with", "extra",  
"ejercicios", "!"]
```

If you've completed all of the exercises so far, congratulations! There are supplementary exercises you can do now. Ask a TA to show you how to get access to them.